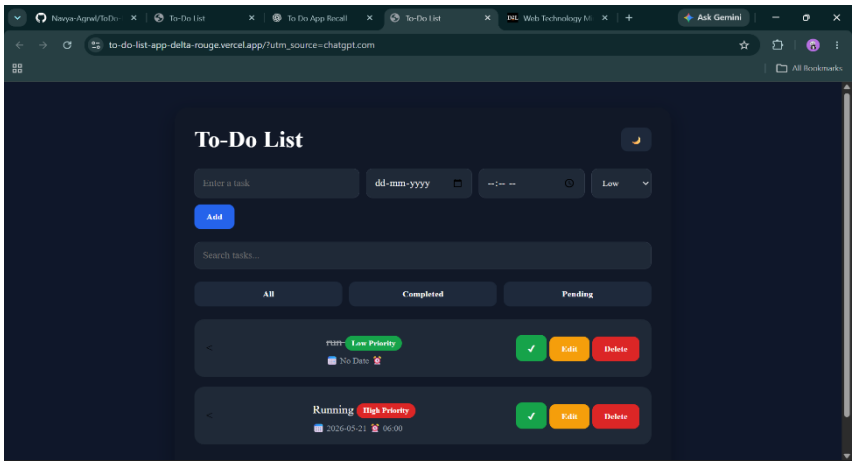
PROJECT DEMONSTRATION REPORT

TO-DO LIST WEB APPLICATION

A responsive task-management application built with HTML5, CSS3 and Vanilla JavaScript



Submitted by	Navya Aggarwal
Project type	Front-end Web Development
Technology stack	HTML5 • CSS3 • JavaScript
Live application	https://to-do-list-app-delta-rouge.vercel.app/
Academic year	2026

1. Project Overview

The To-Do List Web Application is a browser-based tool designed to help users organize everyday activities. Users can create tasks, assign a priority, add a due date and time, update or remove tasks, mark tasks as completed, and find tasks using search and status filters.

The interface is designed to be simple and responsive. It includes a theme-toggle control, and task information is stored in the browser using Local Storage so saved tasks can be restored when the page is opened again in the same browser.

Project objectives

• Create and manage daily tasks through a simple interface.

• Assign Low, Medium or High priority to each task.

• Add due-date and time information.

• Search tasks and filter by completion status.

• Preserve task data across page refreshes using browser storage.

• Provide a layout that adapts to desktop and mobile screens.

Live project: https://to-do-list-app-delta-rouge.vercel.app/

2. Technologies and Tools Used

Technology	Purpose
HTML5	Defines the page structure, headings, input fields, dropdown, buttons and task-list container.
CSS3	Controls layout, spacing, colours, task cards, buttons, dark theme and responsive presentation.
Vanilla JavaScript	Implements task creation, editing, completion, deletion, searching, filters and theme interaction.
DOM API	Finds page elements and creates, updates or removes task elements dynamically.
Local Storage + JSON	Stores task data in the browser and converts data between JavaScript objects and JSON text.
Vercel	Hosts the application online so it can be accessed through a live URL.

3. Project Structure and Application Flow

The project separates structure, presentation and behaviour into different files:

index.html – contains the page structure and links the CSS and JavaScript files.
style.css – contains visual styling, layout and responsive rules.
script.js – contains task data, event listeners and application functions.
README.md – describes the project, features, technology stack and running instructions.

Main task data structure

Each task is represented as a JavaScript object with properties such as **text**, **completed**, **date**, **time** and **priority**. Multiple task objects are stored together in the **tasks** array.

Application flow

Step	What happens
1. User input	The user enters task details and selects Add.
2. Create task	addTask() validates the task text and creates a task object.
3. Update data	The object is added to the tasks array and saved.
4. Display	renderTask() creates the task interface and attaches its buttons.
5. Restore	On page load, saved tasks are read from Local Storage and rendered again.

4. Module 1 – Add a New Task

The input module contains a task text field, date picker, time picker, priority dropdown and Add button. The user can enter a task title, optionally choose a date and time, and select Low, Medium or High priority.

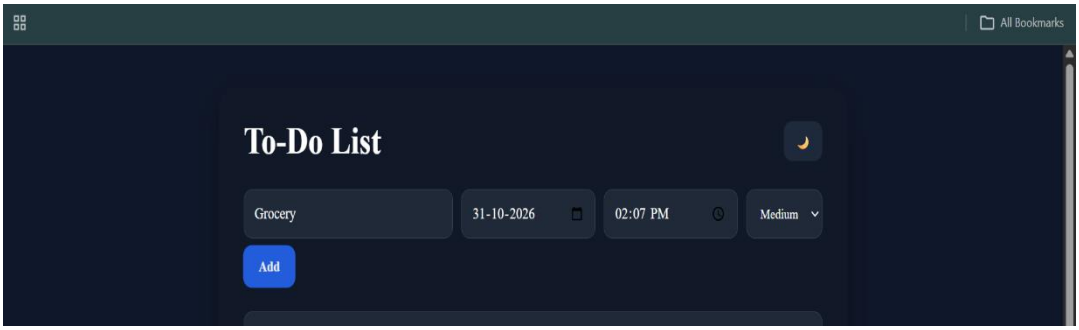


Figure 1. Task entry form with sample task details.

Implementation: JavaScript reads values from the input elements using their IDs. The addTask() function trims the task text, checks that it is not empty, creates a task object, adds it to the tasks array using push(), calls saveTasks(), and displays the task using renderTask().

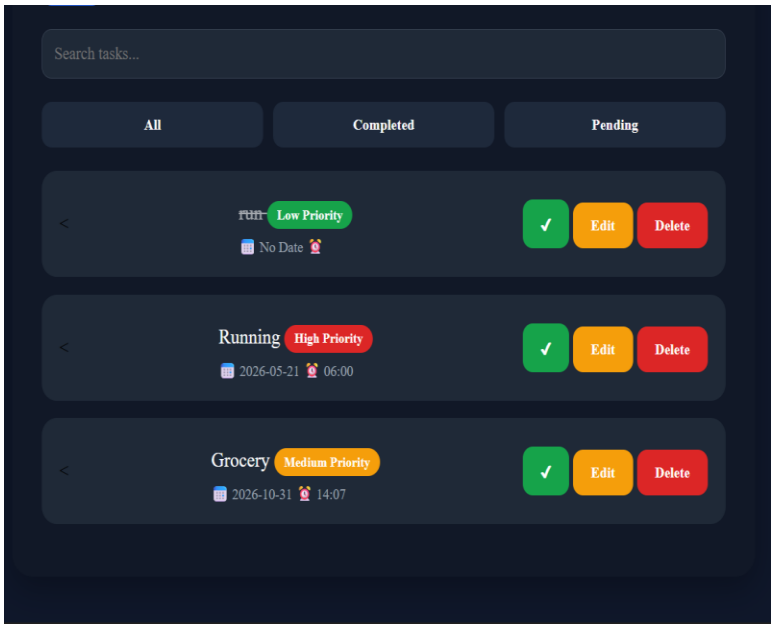
5. Module 2 – Task Management

Each displayed task includes controls to mark it as completed, edit its description or delete it. A priority label and selected date/time are also shown when available.

Figure 2. Task cards showing priority, due-date information and action buttons.

• **Complete:** toggles the task.completed value between true and false and updates its completed visual style.
• **Edit:** uses prompt() to request an updated task description, then updates the task object and displayed text.
• **Delete:** removes the task from the displayed list and updates the tasks array before saving.

The task buttons use addEventListener() to respond to click events. DOM manipulation is used to update the task list dynamically.



6. Module 3 – Search and Status Filters

The search field helps users find tasks by their text. The All, Completed and Pending buttons display tasks according to their completion state.

JavaScript uses the array filter() method to select matching tasks. The search function converts both the search query and task text to lowercase and uses includes() to check whether the task contains the entered text. This makes the search case-insensitive.

Filter logic: All displays every task; Completed selects tasks where completed is true; Pending selects tasks where completed is false.

The search field is connected to the input event, so results refresh as the user types. The filter buttons are connected to click events.

7. Module 4 – Local Storage and Data Persistence

Local Storage is a browser feature that stores key-value data on the user's device. In this project, the task array is saved under the key **tasks**. Saved tasks can remain available after refreshing or reopening the application in the same browser, provided the browser's site data has not been cleared.

The project uses these operations:
• **JSON.stringify(tasks)** converts the JavaScript array into a JSON string.
• **localStorage.setItem("tasks", ...)** saves that string in the browser.
• **localStorage.getItem("tasks")** retrieves the saved string.
• **JSON.parse(...)** converts the retrieved string back into JavaScript data.

The tasks array is initialized from Local Storage when the script starts. The DOMContentLoaded event then calls loadTasks(), which renders saved tasks on the page.

8. Module 5 – Theme Toggle and Responsive Layout

The moon button toggles the dark-mode class on the document body. CSS rules associated with that class change the page's appearance. In the supplied JavaScript, the selected theme is not saved in Local Storage, so the theme may return to its default after a refresh.

Figure 3. Mobile-sized view of the application interface.

The mobile screenshots show the interface in a narrow viewport, with controls and task actions arranged vertically for easier use on a smaller screen. Responsive behaviour is provided by CSS layout rules and media queries in the stylesheet.



9. Demonstration Procedure and Testing

The following steps can be followed during the project demonstration:

1. Open the deployed application in a web browser.

2. Enter a sample task, choose a date and time, select a priority, and click Add.

3. Add two or three more tasks with different priorities.

4. Mark one task as completed and show the Completed and Pending filters.

5. Use the search box to find a task by its title.

6. Edit a task and then delete another task.

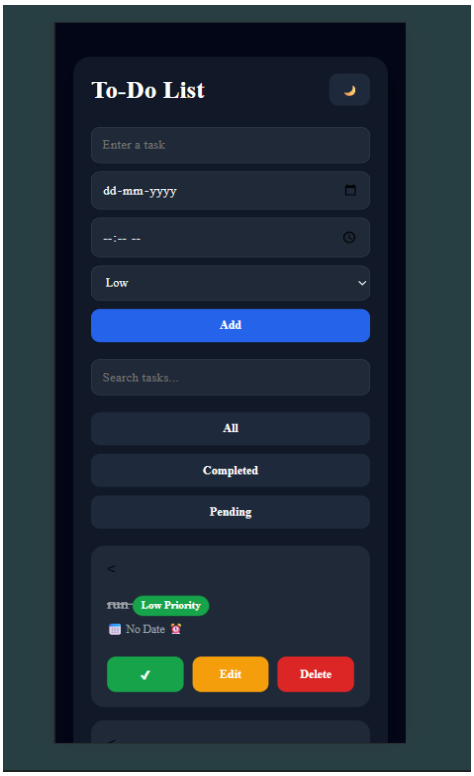
7. Refresh the page to demonstrate that task data is restored from Local Storage.

8. Toggle the theme and use mobile device emulation to show the responsive layout.

Main desktop interface

Figure 4. Main desktop interface of the deployed To-Do List application.

The supplied screenshots document the desktop layout, task entry fields, task cards, action buttons and mobile viewport. They are included as visual evidence of the project interface.



10. Learning Outcomes

• Understanding the separation of HTML structure, CSS presentation and JavaScript behaviour.

• Using DOM methods such as getElementById(), createElement(), querySelector() and appendChild().

• Handling user actions through addEventListener().

• Working with JavaScript arrays, objects, functions, conditions and loops.

• Using array methods such as push(), filter() and forEach().

• Storing and retrieving browser data with Local Storage and JSON.

11. Conclusion

The To-Do List Web Application demonstrates how core front-end web technologies can be combined to build a useful interactive application. It provides task creation, priority selection, due-date and time entry, completion tracking, editing, deletion, searching and status-based filtering. Local Storage supports persistence of task data within the browser, while CSS provides the visual design and responsive layout.

Through this project, practical experience was gained in HTML page structure, CSS styling, JavaScript logic, DOM manipulation, event handling, array and object operations, and browser storage.

Current scope and limitations

• Task data is stored in the browser's Local Storage; it is not synchronized to an online database or across different devices.
• The date and time fields record and display due-date information. The supplied JavaScript does not implement scheduled notifications or automatic reminder alerts.
• The dark-mode toggle changes the page class, but the theme preference is not persisted after a refresh.
• The project is a front-end application and does not include user accounts or server-side task storage.

12. Project Information

Project: To-Do List Web Application
Author: Navya Aggarwal
Technologies: HTML5, CSS3, Vanilla JavaScript
Deployment: Vercel
Live URL: https://to-do-list-app-delta-rouge.vercel.app/

— End of Report —